

Runtime Directive Elements

Article • 10/20/2022

The runtime directives (rd.xml) file format supports the following runtime directive elements. See [Runtime Directives \(rd.xml\) Configuration File Reference](#) for a hierarchical representation.

<Application>

Applies runtime reflection policy to all types used by the app, and serves as a container for application-wide types and type members whose metadata is available for reflection at run time. This is a child of the [<Directives>](#) element.

<Assembly>

Applies runtime policy to all the types in an assembly. This is a child of the [<Application>](#) and [<Library>](#) elements.

<AttributeImplies>

If its containing [<Type>](#) directive is an attribute, applies runtime policy to code elements to which that attribute is applied.

<Directives>

The root element in every runtime directives file for .NET Native. Its child elements are [<Application>](#) and [<Library>](#).

<Event>

Applies runtime policy to an event. This is a child of the [<Type>](#) and [<TypeInstantiation>](#) elements.

<Field>

Applies runtime policy to a field. This is a child of the [<Type>](#) and [<TypeInstantiation>](#) elements.

<GenericParameter>

Applies runtime policy to the parameter type of a generic type or method.

<ImpliesType>

Applies runtime policy to a type, if that policy has been applied to the containing type or method.

<Library>

Applies runtime policy to all the types in an assembly. This is a child of the [<Application>](#) and [<Library>](#) elements.

[<Method>](#)

Applies runtime policy to a method. This is a child of the [<Type>](#) and [<TypeInstantiation>](#) elements.

[<MethodInstantiation>](#)

Applies runtime policy to a constructed generic method. This is a child of the [<Type>](#) and [<TypeInstantiation>](#) elements.

[<Namespace>](#)

Applies runtime policy to all the types in a namespace.

[<Parameter>](#)

Applies runtime policy to the type of the argument passed to a method.

[<Property>](#)

Applies runtime policy to a property. This is a child of the [<Type>](#) and [<TypeInstantiation>](#) elements.

[<Subtypes>](#)

Applies runtime policy to all classes inherited from the containing type.

[<Type>](#)

Applies runtime policy to a type.

[<TypeInstantiation>](#)

Applies runtime policy to a constructed generic type.

[<TypeParameter>](#)

Applies runtime policy to the type represented by a [Type](#) argument passed to a method.

See also

- [rd.xml Configuration File Reference](#)

<Application> Element (.NET Native)

Article • 10/20/2022

Serves as a container for application-wide types and type members whose metadata is available for reflection at run time, and applies runtime reflection policy to all the program elements in an app.

<Directives> Element <Application> Element (rd.xml)

Syntax

XML

```
<Application Activate="policy_setting"
    Browse="policy_setting"
    Dynamic="policy_setting"
    Serialize="policy_setting"
    DataContractSerializer="policy_setting"
    DataContractJsonSerializer="policy_setting"
    XmlSerializer="policy_setting"
    MarshalObject="policy_setting"
    MarshalDelegate="policy_setting"
    MarshalStructure="policy_setting" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements. In the Child Elements table, policy refers to the kind of metadata that is made available for particular program elements at run time.

Attributes

Attribute	Attribute type	Description
Activate	Reflection	Optional attribute. Controls runtime access to constructors to enable activation of instances.
Browse	Reflection	Optional attribute. Controls querying for information about or enumerating the types, but does not enable any dynamic access at run time.

Attribute	Attribute type	Description
<code>Dynamic</code>	Reflection	Optional attribute. Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming.
<code>Serialize</code>	Serialization	Optional attribute. Controls runtime access to constructors, fields, and properties, to enable type instances to be serialized and deserialized by libraries such as the Newtonsoft JSON serializer.
<code>DataContractSerializer</code>	Serialization	Optional Attribute. Controls policy for serialization that uses the System.Runtime.Serialization.DataContractSerializer class.
<code>DataContractJsonSerializer</code>	Serialization	Optional Attribute. Controls policy for JSON serialization that uses the System.Runtime.Serialization.Json.DataContractJsonSerializer class.
<code>XmlSerializer</code>	Serialization	Optional Attribute. Controls policy for XML serialization that uses the System.Xml.Serialization.XmlSerializer class.
<code>MarshalObject</code>	Interop	Optional Attribute. Controls policy for marshaling reference types to Windows Runtime and COM.
<code>MarshalDelegate</code>	Interop	Optional Attribute. Controls policy for marshaling delegate types as function pointers to native code.
<code>MarshalStructure</code>	Interop	Optional Attribute. Controls policy for marshaling structures to native code.

All attributes

Value	Description
<i>policy_setting</i>	The setting for this policy to apply to the types in the app. Possible values are <code>All</code> , <code>Auto</code> , <code>Excluded</code> , <code>Public</code> , <code>PublicAndInternal</code> , <code>Required Public</code> , <code>Required PublicAndInternal</code> , and <code>Required All</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

Element	Description
<code><Assembly></code>	Applies policy to all the types in a particular assembly.

Element	Description
<code><Namespace></code>	Applies policy to all the types in a particular namespace.
<code><Type></code>	Applies policy to a particular type, such as a class or structure.
<code><TypeInstantiation></code>	Applies policy to a constructed generic type. For example, a <code><TypeInstantiation></code> element could be used to define policy for a <code>List<String></code> type.
<code><Method></code>	Applies policy to a method on a particular type.
<code><MethodInstantiation></code>	Applies policy to a constructed generic method.
<code><Property></code>	Applies policy to a property on a particular type.
<code><Field></code>	Applies policy to a field on a particular type.
<code><Event></code>	Applies policy to an event on a particular type.

Parent Elements

Element	Description
<code><Directives></code>	The root element of a runtime directives file.

Remarks

The `<Directives>` element can contain zero or one `<Application>` element. Multiple `<Application>` elements in a single reflection directives file are not supported.

The `<Application>` element can be used in one of two ways:

- As a container to define the program elements whose metadata is needed at run time. In this case, the `<Application>` element need not have any attributes. At compile time, compiler tools search all libraries, including .NET Framework core libraries, for program elements identified by child elements of the `<Application>` element. In contrast, compiler tools search only the library designated by the `<Library>` element for program elements identified by the child elements of `<Library>`.
- As an element that sets application-wide policy for reflection, serialization, and interop. The attributes of the `<Application>` element define application-wide policy, which may be overridden by the child elements defined by the `<Application>` or `<Library>` element.

See also

- [<Library> Element](#)
- [<Directives> Element](#)
- [Runtime Directive Elements](#)
- [Runtime Directives \(rd.xml\) Configuration File Reference](#)

<Assembly> Element (.NET Native)

Article • 10/20/2022

Applies runtime reflection policy to all the types in a specified assembly.

Syntax

XML

```
<Assembly Name="assembly_name"
  Activate="policy_setting"
  Browse="policy_setting"
  Dynamic="policy_setting"
  Serialize="policy_setting"
  DataContractSerializer="policy_setting"
  DataContractJsonSerializer="policy_setting"
  XmlSerializer="policy_setting"
  MarshalObject="policy_setting"
  MarshalDelegate="policy_setting"
  MarshalStructure="policy_setting" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
Name	General	Required attribute. Specifies the simple name of an assembly.
Activate	Reflection	Optional attribute. Controls runtime access to constructors to enable activation of instances.
Browse	Reflection	Optional attribute. Controls querying for information about or enumerating the types in the assembly, but does not enable any dynamic access at run time.
Dynamic	Reflection	Optional attribute. Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming.

Attribute	Attribute type	Description
<code>Serialize</code>	Serialization	Optional attribute. Controls runtime access to constructors, fields, and properties, to enable type instances to be serialized and deserialized by libraries such as the Newtonsoft JSON serializer.
<code>DataContractSerializer</code>	Serialization	Optional attribute. Controls policy for serialization that uses the System.Runtime.Serialization.DataContractSerializer class.
<code>DataContractJsonSerializer</code>	Serialization	Optional attribute. Controls policy for JSON serialization that uses the System.Runtime.Serialization.Json.DataContractJsonSerializer class.
<code>XmlSerializer</code>	Serialization	Optional attribute. Controls policy for XML serialization that uses the System.Xml.Serialization.XmlSerializer class.
<code>MarshalObject</code>	Interop	Optional attribute. Controls policy for marshaling reference types to Windows Runtime and COM.
<code>MarshalDelegate</code>	Interop	Optional attribute. Controls policy for marshaling delegate types as function pointers to native code.
<code>MarshalStructure</code>	Interop	Optional attribute. Controls policy for marshaling structures to native code.

Name attribute

Value	Description
<i>assembly_name</i>	The simple name of the assembly, without its file extension. This attribute corresponds to the AssemblyName.Name property. For example, the name of an assembly named Extensions.dll is "Extensions". You can also specify the literal string <code>*Application*</code> to apply policy to all assemblies in your app package, whether those assemblies are loaded or not. <code>*Application*</code> never applies policy to .NET Framework assemblies.

All other attributes

Value	Description
-------	-------------

Value	Description
<i>policy_setting</i>	The setting to apply to this policy type for all types in the assembly. Possible values are <code>All</code> , <code>Auto</code> , <code>Excluded</code> , <code>Public</code> , <code>PublicAndInternal</code> , <code>Required Public</code> , <code>Required PublicAndInternal</code> , and <code>Required All</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

Element	Description
<Namespace>	Applies reflection policy to all types in a child namespace.
<Type>	Applies reflection policy to a type.
<TypeInstantiation>	Applies reflection policy to a constructed generic type.

Parent Elements

Element	Description
<Application>	Serves as a container for application-wide types and type members whose metadata is available for reflection at run time. The <Application> element can have zero, one, or more <code><Assembly></code> elements.
<Library>	Defines the assembly that contains types and type members whose metadata is available for reflection at run time. The <Library> element can have zero or one <code><Assembly></code> element.

Remarks

The `<Assembly>` element defines runtime policy for all the types in an assembly. It differs from the [<Library>](#) element, which specifies a library but depends on its child elements to define runtime reflection policy. The `<Assembly>` element applies to all the types in an assembly unless they are overridden by a child element.

The following example shows how you can apply runtime policy to all the types in assemblies within your app package by assigning the `Name` attribute a value of `"*Application*"`. The `<Assembly>` element must be a child of the [<Application>](#) element.

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
```

```
<Assembly Name="*Application*" Dynamic="Required All" />
</Application>
</Directives>
```

The `Activate`, `Browse`, `Dynamic`, and `Serialize` attributes are all optional. However, the `<Assembly>` element must contain at least one of these attributes.

See also

- [Runtime Directive Policy Settings](#)
- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Elements](#)

<AttributeImplies> Element (.NET Native)

Article • 10/20/2022

Defines policy for code elements the containing attribute is applied to.

Syntax

XML

```
<AttributeImplies Activate="policy_type"
  Browse="policy_type"
  Dynamic="policy_type"
  Serialize="policy_type"
  DataContractSerializer="policy_setting"
  DataContractJsonSerializer="policy_setting"
  XmlSerializer="policy_setting"
  MarshalObject="policy_setting"
  MarshalDelegate="policy_setting"
  MarshalStructure="policy_setting" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
Activate	Reflection	Optional attribute. Controls runtime access to constructors to enable activation of instances.
Browse	Reflection	Optional attribute. Controls querying for information about program elements, but does not enable any runtime access.
Dynamic	Reflection	Optional attribute. Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming.

Attribute	Attribute type	Description
<code>Serialize</code>	Serialization	Optional attribute. Controls runtime access to constructors, fields, and properties, to enable type instances to be serialized and deserialized by libraries such as the Newtonsoft JSON serializer.
<code>DataContractSerializer</code>	Serialization	Optional attribute. Controls policy for serialization that uses the System.Runtime.Serialization.DataContractSerializer class.
<code>DataContractJsonSerializer</code>	Serialization	Optional attribute. Controls policy for JSON serialization that uses the System.Runtime.Serialization.Json.DataContractJsonSerializer class.
<code>XmlSerializer</code>	Serialization	Optional attribute. Controls policy for XML serialization that uses the System.Xml.Serialization.XmlSerializer class.
<code>MarshalObject</code>	Interop	Optional attribute. Controls policy for marshaling reference types to Windows Runtime and COM.
<code>MarshalDelegate</code>	Interop	Optional attribute. Controls policy for marshaling delegate types as function pointers to native code.
<code>MarshalStructure</code>	Interop	Optional attribute. Controls policy for marshaling value types to native code.

All attributes

Value	Description
<i>policy_setting</i>	The setting to apply to this policy type. Possible values are <code>All</code> , <code>Auto</code> , <code>Excluded</code> , <code>Public</code> , <code>PublicAndInternal</code> , <code>Required Public</code> , <code>Required PublicAndInternal</code> , and <code>Required All</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

None.

Parent Elements

Element	Description
<Type>	Applies reflection policy to a type and all its members.

Remarks

The `<AttributeImplies>` element is used if its containing type is an attribute (that is, a class derived from [System.Attribute](#)). If the attribute is applied to a particular program element, the policy defined by the `<AttributeImplies>` element applies to that program element.

The reflection, serialization, and interop attributes are all optional, although at least one should be present.

See also

- [<Type> Element](#)
- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Elements](#)
- [Runtime Directive Policy Settings](#)

<Directives> Element (.NET Native)

Article • 10/20/2022

The root element in every runtime directives file for .NET Native.

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
```

Syntax

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">  
  <!-- child elements -->  
</Directives>
```

Attributes

Attribute	Description
<code>xmlns</code>	The XML namespace. Its value is always <code>http://schemas.microsoft.com/netfx/2013/01/metadata</code> .

Child elements

Element	Description
<code><Application></code>	Serves as a container for application-wide types and type members whose metadata is available for reflection.
<code><Library></code>	Defines the assembly whose child types and type members require metadata at run time.

Remarks

Each runtime directives file can contain only one `<Directives>` element.

The `<Directives>` element can contain zero or one `<Application>` element, and zero, one, or more `<Library>` elements.

See also

- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Elements](#)

<Event> Element (.NET Native)

Article • 10/20/2022

Applies runtime reflection policy to an event.

Syntax

XML

```
<Event Name="event_name"  
      Browse="policy_type"  
      Dynamic="policy_type" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
Name	General	Required attribute. Specifies the event name.
Browse	Reflection	Optional attribute. Controls querying for information about or enumerating the event but does not enable any dynamic access at run time.
Dynamic	Reflection	Optional attribute. Controls runtime access to the event to enable dynamic programming. This policy ensures that an event can be handled dynamically at run time.

Name attribute

Value	Description
<i>method_name</i>	The event name. The type of the event is defined by the parent <Type> or <TypeInstantiation> element.

All other attributes

Value	Description
<i>policy_setting</i>	The setting to apply to this policy type for the event. Possible values are <code>Auto</code> , <code>Excluded</code> , <code>Included</code> , and <code>Required</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

None.

Parent Elements

Element	Description
<Type>	Applies reflection policy to a type and all its members.
<TypeInstantiation>	Applies reflection policy to a constructed generic type and all its members.

Remarks

If an event's policy is not explicitly defined, it inherits the runtime policy of its parent element.

See also

- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Elements](#)
- [Runtime Directive Policy Settings](#)

<Field> Element (.NET Native)

Article • 10/20/2022

Applies runtime reflection policy to a field.

Syntax

XML

```
<Field Name="field_name"  
      Browse="policy_type"  
      Dynamic="policy_type"  
      Serialize="policy_type" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
Name	General	Required attribute. Specifies the field name.
Browse	Reflection	Optional attribute. Controls querying for information about or enumerating the field but does not enable any dynamic access at run time.
Dynamic	Reflection	Optional attribute. Controls runtime access to the field to enable dynamic programming. This policy ensures that a field can be set or retrieved dynamically at run time.
Serialize	Serialization	Optional attribute. Controls runtime access to a field to enable type instances to be serialized by libraries such as the Newtonsoft JSON serializer or to be used for data binding.

Name attribute

Value	Description
-------	-------------

Value	Description
<i>method_name</i>	The field name. The type of the field is defined by the parent <Type> or <TypeInstantiation> element.

All other attributes

Value	Description
<i>policy_setting</i>	The setting to apply to this policy type for the field. Possible values are <code>Auto</code> , <code>Excluded</code> , <code>Included</code> , and <code>Required</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

None.

Parent Elements

Element	Description
<Type>	Applies reflection policy to a type and all its members.
<TypeInstantiation>	Applies reflection policy to a constructed generic type and all its members.

Remarks

If a field's policy is not explicitly defined, it inherits the runtime policy of its parent element.

See also

- [Runtime Directive Elements](#)
- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Policy Settings](#)

<GenericParameter> Element (.NET Native)

Article • 10/20/2022

Applies policy to the parameter type of a generic type or method.

Syntax

XML

```
<GenericParameter Name="generic_parameter_name"
  Activate="policy_type"
  Browse="policy_type"
  Dynamic="policy_type"
  Serialize="policy_type"
  DataContractSerializer="policy_type"
  DataContractJsonSerializer="policy_type"
  XmlSerializer="policy_type"
  MarshalObject="policy_type"
  MarshalDelegate="policy_type"
  MarshalStructure="policy_type" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
<code>Name</code>	General	Required attribute. The name of the generic parameter. For example, for the generic delegate <code>Func<T1,T2,TResult></code> , the value of the <code>Name</code> attribute is "TResult" to apply runtime policy to the delegate's return value.
<code>Activate</code>	Reflection	Optional attribute. Controls runtime access to constructors to enable activation of instances.
<code>Browse</code>	Reflection	Optional attribute. Controls querying for information about program elements, but does not enable any runtime access.

Attribute	Attribute type	Description
<code>Dynamic</code>	Reflection	Optional attribute. Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming.
<code>Serialize</code>	Serialization	Optional attribute. Controls runtime access to constructors, fields, and properties, to enable type instances to be serialized and deserialized by libraries such as the Newtonsoft JSON serializer.
<code>DataContractSerializer</code>	Serialization	Optional attribute. Controls policy for serialization that uses the System.Runtime.Serialization.DataContractSerializer class.
<code>DataContractJsonSerializer</code>	Serialization	Optional attribute. Controls policy for JSON serialization that uses the System.Runtime.Serialization.Json.DataContractJsonSerializer class.
<code>XmlSerializer</code>	Serialization	Optional attribute. Controls policy for XML serialization that uses the System.Xml.Serialization.XmlSerializer class.
<code>MarshalObject</code>	Interop	Optional attribute. Controls policy for marshaling reference types to Windows Runtime and COM.
<code>MarshalDelegate</code>	Interop	Optional attribute. Controls policy for marshaling delegate types as function pointers to native code.
<code>MarshalStructure</code>	Interop	Optional attribute. Controls policy for marshaling value types to native code.

Name attribute

Value	Description
<code>generic_parameter_name</code>	Required attribute. The name of the generic type parameter. For example, for the generic delegate Func<T1,T2,TResult> , a <code>generic_parameter_name</code> value of "TResult" applies runtime policy to the delegate's return value.

All other attributes

Value	Description
-------	-------------

Value	Description
<i>policy_setting</i>	The setting to apply to this policy type. Possible values are <code>All</code> , <code>Public</code> , <code>PublicAndInternal</code> , <code>Required Public</code> , <code>Required PublicAndInternal</code> , and <code>Required All</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

None.

Parent Elements

Element	Description
<Method>	Applies runtime reflection policy to a constructor or method.
<Type>	Applies runtime reflection policy to a particular type, such as a class or structure.

Remarks

The `<GenericParameter>` element is a child of either the [<Method>](#) or [<Type>](#) element and is used to apply policy to a particular generic type parameter, which is specified by its name in the generic type or method signature.

The `<GenericParameter>` element is most useful when used with serializers. The following example uses the `<GenericParameter>` element to apply policy to the type `T` in calls to the Newtonsoft JSON serializer's [JsonConvert.DeserializeObject<T>\(String\)](#) [↗](#) method overloads.

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Type Name="Newtonsoft.Json.JsonConvert" >
    <Method Name="DeserializeObject{T}">
      <GenericParameter Name="T" Serialize="Required All" />
    </Method>
  </Type>
</Directives>
```

See also

- [<Method> Element](#)

- [<Type> Element](#)
- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Policy Settings](#)
- [Runtime Directive Elements](#)

<ImpliesType> Element (.NET Native)

Article • 10/20/2022

Applies policy to a type, if that policy has been applied to the containing type or method.

Syntax

XML

```
<ImpliesType Name="type_name"
  Activate="policy_type"
  Browse="policy_type"
  Dynamic="policy_type"
  Serialize="policy_type"
  DataContractSerializer="policy_setting"
  DataContractJsonSerializer="policy_setting"
  XmlSerializer="policy_setting"
  MarshalObject="policy_setting"
  MarshalDelegate="policy_setting"
  MarshalStructure="policy_setting" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
Name	General	Required attribute. Specifies the type name.
Activate	Reflection	Optional attribute. Controls runtime access to constructors to enable activation of instances.
Browse	Reflection	Optional attribute. Controls querying for information about program elements, but does not enable any runtime access.
Dynamic	Reflection	Optional attribute. Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming.

Attribute	Attribute type	Description
<code>Serialize</code>	Serialization	Optional attribute. Controls runtime access to constructors, fields, and properties, to enable type instances to be serialized and deserialized by libraries such as the Newtonsoft JSON serializer.
<code>DataContractSerializer</code>	Serialization	Optional attribute. Controls policy for serialization that uses the System.Runtime.Serialization.DataContractSerializer class.
<code>DataContractJsonSerializer</code>	Serialization	Optional attribute. Controls policy for JSON serialization that uses the System.Runtime.Serialization.Json.DataContractJsonSerializer class.
<code>XmlSerializer</code>	Serialization	Optional attribute. Controls policy for XML serialization that uses the System.Xml.Serialization.XmlSerializer class.
<code>MarshalObject</code>	Interop	Optional attribute. Controls policy for marshaling reference types to Windows Runtime and COM.
<code>MarshalDelegate</code>	Interop	Optional attribute. Controls policy for marshaling delegate types as function pointers to native code.
<code>MarshalStructure</code>	Interop	Optional attribute. Controls policy for marshaling value types to native code.

Name attribute

Value	Description
<i>type_name</i>	The type name. If the type represented by this <code><ImpliesType></code> element is located in the same namespace as its containing <code><Type></code> element, <i>type_name</i> can include the name of the type without its namespace. Otherwise, <i>type_name</i> must include the fully qualified type name.

All other attributes

Value	Description
<i>policy_setting</i>	The setting to apply to this policy type. Possible values are <code>All</code> , <code>Auto</code> , <code>Excluded</code> , <code>Public</code> , <code>PublicAndInternal</code> , <code>Required Public</code> , <code>Required PublicAndInternal</code> , and <code>Required All</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

None.

Parent Elements

Element	Description
<code><Type></code>	Applies reflection policy to a type and all its members.
<code><TypeInstantiation></code>	Applies reflection policy to a constructed generic type and all its members.
<code><Method></code>	Applies reflection policy to a method.

Remarks

The `<ImpliesType>` element is primarily intended for use by libraries. It addresses the following scenario:

- If a routine needs to reflect on one type, it necessarily needs to reflect on a second type.
- The metadata for the implied instantiation of the second type is otherwise unavailable, because static analysis doesn't indicate that it's necessary.

Most commonly, the two types are generic instantiations with shared type arguments.

The `<ImpliesType>` element was defined with the assumption that a need for reflection on the type specified by its parent element implies a need for reflection on the type specified by the `<ImpliesType>` element. For example, the following reflection directives apply to two types, `Explicit<T>` and `Implicit<T>`.

XML

```
<Type Name="Explicit{ET}">
  <ImpliesType Name="Implicit{ET}" Dynamic="Required Public" />
</Type>
```

This directive has no effect unless an instantiation of `Explicit` has a defined `Dynamic` policy setting. For example, if that's the case for `Explicit<Int32>`, `Implicit<Int32>` is instantiated with its public members rooted, and their metadata is made accessible for dynamic programming.

The following is a real-world example that applies to at least one serializer. The directives capture the requirement that reflecting on something typed as `IList<something>` also involves reflecting on the corresponding `List<something>` type without requiring any per-application annotation.

XML

```
<Type Name="System.Collections.Generic.IList{T}">
  <ImpliesType Name="System.Collections.Generic.List{T}" Serialize="Public"
/>
</Type>
```

The `<ImpliesType>` element can also appear within a `<Method>` element, because in some cases instantiating a generic method implies reflecting on a type instantiation. For example, imagine a generic method `IEnumerable<T> MakeEnumerable<T>(string spelling, T defaultValue)` that a given library will access dynamically along with the associated `List<T>` and `Array` types. This can be expressed as:

XML

```
<Type Name="MyType">
  <Method Name="MakeEnumerable{T}" Signature="(System.String, T)"
Dynamic="Included">
    <ImpliesType Name="T[]" Dynamic="Public" />
    <ImpliesType Name="System.Collections.Generic.List{T}"
Dynamic="Public" />
  </Method>
</Type>
```

See also

- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Elements](#)
- [Runtime Directive Policy Settings](#)

<Library> Element (.NET Native)

Article • 10/20/2022

Defines the assembly that contains types and type members whose metadata is available for reflection at run time.

<Directives> Element <Library> Element

Syntax

XML

```
<Library Name="assembly_name" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Description
Name	Required attribute. Specifies the name of an assembly. Child elements of this <Library> element define the runtime reflection policy for types and type members found in this assembly.

Name attribute

Value	Description
<i>assembly_name</i>	The simple name of the assembly, without its file extension. This attribute corresponds to the AssemblyName.Name property. For example, the name of an assembly named Extensions.dll is "Extensions". See the Remarks section for a special form of <i>assembly_name</i> that supports conditional inclusion of metadata from the assembly.

Child Elements

Element	Description
---------	-------------

Element	Description
<code><Assembly></code>	Applies policy to all the types in a particular assembly.
<code><Namespace></code>	Applies policy to all the types in a particular namespace.
<code><Type></code>	Applies policy to a particular type, such as a class or structure.
<code><TypeInstantiation></code>	Applies policy to a constructed generic type. For example, a <code><TypeInstantiation></code> element could be used to define policy for a <code>List<String></code> type.

Parent Elements

Element	Description
<code><Directives></code>	The root element of a runtime directives file.

Remarks

The `<Directives>` element can contain zero, one, or more `<Library>` elements.

The `<Library>` element serves as a container to define the program elements whose metadata is needed at run time; this element doesn't express policy. At compile time, compiler tools search only the library designated by the `<Library>` element for program elements identified by its child elements. In contrast, compiler tools search all libraries, including .NET Framework core libraries, for program elements identified by child elements of the `<Application>` element.

`<Library>` directives may be conditionally utilized. If the name of the `<Library>` element starts and ends with an asterisk (*), the `<Library>` directive has an effect only if the assembly specified between the asterisks is referenced by the app. For example, the following runtime directive applies only if the Utilities.dll assembly is referenced by the app.

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Library Name="*Utilities*">
    ...
  </Library>
</Directives>
```

See also

- [<Application> Element](#)
- [<Directives> Element](#)
- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Elements](#)

<Method> Element (.NET Native)

Article • 10/20/2022

Applies runtime reflection policy to a constructor or method.

Syntax

XML

```
<Method Name="method_name"
        Signature="method_signature"
        Browse="policy_type"
        Dynamic="policy_type" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

 Expand table

Attribute	Attribute type	Description
Name	General	Required attribute. Specifies the method name.
Signature	General	Optional attribute. Specifies the method signature. If multiple parameters are present, they are separated by commas. For example, the following <code><Method></code> element defines policy for the <code>ToString(String, IFormatProvider)</code> method. <pre><Type Name="System.DateTime"> <Method Name="ToString" Signature="System.String,System.IFormatProvider" Dynamic="Required" /> </Type></pre> If the attribute is absent, the runtime directive applies to all overloads of the method.
Browse	Reflection	Optional attribute. Controls querying for information about or enumerating a method but does not enable any dynamic invocation at run time.

Attribute	Attribute type	Description
<code>Dynamic</code>	Reflection	Optional attribute. Controls runtime access to a constructor or method to enable dynamic programming. This policy ensures that a member can be invoked dynamically at run time.

Name attribute

 Expand table

Value	Description
<i>method_name</i>	The method name. The type of the method is defined by the parent <code><Type></code> or <code><TypeInstantiation></code> element.

Signature attribute

 Expand table

Value	Description
<i>method_signature</i>	The parameter types that form the method signature. Multiple parameters are separated by commas, for example, <code>"System.String,System.Int32,System.Int32)"</code> . Parameter type names should be fully qualified.

All other attributes

 Expand table

Value	Description
<i>policy_setting</i>	The setting to apply to this policy type. Possible values are <code>Auto</code> , <code>Excluded</code> , <code>Included</code> , and <code>Required</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

 Expand table

Element	Description
<Parameter>	Applies policy to the type of the argument passed to a method.
<GenericParameter>	Applies policy to the parameter type of a generic type or method.
<ImpliesType>	Applies policy to a type, if that policy has been applied to the method represented by the containing <Method> element.
<TypeParameter>	Applies policy to the type represented by a Type argument passed to a method.

Parent Elements

[Expand table](#)

Element	Description
<Type>	Applies reflection policy to a type and all its members.
<TypeInstantiation>	Applies reflection policy to a constructed generic type and all its members.

Remarks

A [<Method>](#) element of a generic method applies its policy to all instantiations that do not have their own policy.

You can use the [Signature](#) attribute to specify policy for a particular method overload. Otherwise, if the [Signature](#) attribute is absent, the runtime directive applies to all overloads of the method.

You cannot define the runtime reflection policy for a constructor by using the [<Method>](#) element. Instead, use the [Activate](#) attribute of the [<Assembly>](#), [<Namespace>](#), [<Type>](#), or [<TypeInstantiation>](#) element.

Example

The [Stringify](#) method in the following example is a general-purpose formatting method that uses reflection to convert an object to its string representation. In addition to calling the object's default [ToString](#) method, the method can produce a formatted result string by passing an object's [ToString](#) method a format string, an [IFormatProvider](#)

implementation, or both. It can also call one of the [Convert.ToString](#) overloads that converts a number to its binary, hexadecimal, or octal representation.

C#

```
public class Stringify
{
    public static string ConvertToString(Object[] obj)
    {
        if (obj == null)
            throw new NullReferenceException("The obj parameter cannot be null.");

        if (obj.Length == 0) return String.Empty;

        if (obj[0].GetType() == typeof(String))
            return obj[0] as string;

        if (obj.Length == 1) return obj[0].ToString();

        if (obj.Length > 3)
            throw new ArgumentOutOfRangeException("The array can have from zero to three elements.");

        string retval = "";

        // Parameters indicate either a format specifier, numeric base, or
        // format provider,
        // or a format specifier with an IFormatProvider.

        // A string as the first parameter indicates a format specifier.
        if (obj[1].GetType() == typeof(String)) {
            Type t = obj[0].GetType();
            if (obj.Length == 2)
            {
                MethodInfo m = t.GetRuntimeMethod("ToString", new Type[] {
                    typeof(String) });
                retval = m.Invoke(obj[0], new object[] { obj[1] }).ToString();
            }
            else
            {
                MethodInfo m = t.GetRuntimeMethod("ToString", new Type[] {
                    typeof(String), obj[2].GetType() });
                retval = m.Invoke(obj[0], new object[] { obj[1], obj[2]
                }).ToString();
            }
        }
        else if (obj[1] is IFormatProvider)
        {
            Type t = obj[0].GetType();
            MethodInfo m = t.GetRuntimeMethod("ToString", new Type[] {
                obj[1].GetType() } );
            retval = m.Invoke(obj[0], new object[] { obj[1] }).ToString();
        }
    }
}
```

```

        // The second parameter is a base, so call Convert.ToString(number,
        int).
        else {
            Type t = typeof(Convert);
            MethodInfo m = t.GetRuntimeMethod("ToString", new Type[] {
            obj[0].GetType(), obj[1].GetType() } );
            retval = m.Invoke(null, obj).ToString();
        }
        return retval;
    }
}

```

The `Stringify` method can be called by code like the following:

C#

```

public class Stringify
{
    public static string ConvertToString(Object[] obj)
    {
        if (obj == null)
            throw new NullReferenceException("The obj parameter cannot be
            null.");

        if (obj.Length == 0) return String.Empty;

        if (obj[0].GetType() == typeof(String))
            return obj[0] as string;

        if (obj.Length == 1) return obj[0].ToString();

        if (obj.Length > 3)
            throw new ArgumentOutOfRangeException("The array can have from zero
            to three elements.");

        string retval = "";

        // Parameters indicate either a format specifier, numeric base, or
        // format provider,
        // or a format specifier with an IFormatProvider.

        // A string as the first parameter indicates a format specifier.
        if (obj[1].GetType() == typeof(String)) {
            Type t = obj[0].GetType();
            if (obj.Length == 2)
            {
                MethodInfo m = t.GetRuntimeMethod("ToString", new Type[] {
                typeof(String) });
                retval = m.Invoke(obj[0], new object[] { obj[1] }).ToString();
            }
            else
            {
                MethodInfo m = t.GetRuntimeMethod("ToString", new Type[] {

```

```

typeof(String), obj[2].GetType() });
        retval = m.Invoke(obj[0], new object[] { obj[1], obj[2]
    }).ToString();
    }
}
else if (obj[1] is IFormatProvider)
{
    Type t = obj[0].GetType();
    MethodInfo m = t.GetRuntimeMethod("ToString", new Type[] {
obj[1].GetType() } );
    retval = m.Invoke(obj[0], new object[] { obj[1] }).ToString();
}
// The second parameter is a base, so call Convert.ToString(number,
int).
else {
    Type t = typeof(Convert);
    MethodInfo m = t.GetRuntimeMethod("ToString", new Type[] {
obj[0].GetType(), obj[1].GetType() } );
    retval = m.Invoke(null, obj).ToString();
}
return retval;
}
}

```

However, when compiled with .NET Native, the example can throw a number of exceptions at runtime, including [NullReferenceException](#) and [MissingRuntimeArtifactException](#) exceptions. This occurs because the `Stringify` method is intended primarily to support dynamically formatting the primitive types in the .NET Framework Class Library. However, their metadata is not made available by the default directives file. Even when their metadata is made available, however, the example throws [MissingRuntimeArtifactException](#) exceptions because the appropriate `ToString` implementations have not been included in the native code.

These exceptions can all be eliminated by using the `<Type>` element to define the types whose metadata must be present, and by adding `<Method>` elements to ensure that the implementation of method overloads that can be called dynamically is also present. The following is the default.rd.xml file that eliminates these exceptions and allows the example to execute without error.

XML

```

<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
    <Assembly Name="*Application*" Dynamic="Required All" />

    <Type Name = "System.Convert" Browse="Required Public"
Dynamic="Required Public" >
      <Method Name="ToString"      Dynamic ="Required" />
    </Type>
  </Application>
</Directives>

```

```

<Type Name="System.Double" Browse="Required Public">
  <Method Name="ToString" Dynamic="Required" />
</Type>
<Type Name ="System.Int32" Browse="Required Public" >
  <Method Name="ToString" Dynamic="Required" />
</Type>
<Type Name ="System.Int64" Browse="Required Public" >
  <Method Name="ToString" Dynamic="Required" />
</Type>
<Namespace Name="System" >
  <Type Name="Byte" Browse="Required Public" >
    <Method Name="ToString" Dynamic="Required" />
  </Type>
  <Type Name="DateTime" Browse="Required Public" >
    <Method Name="ToString" Dynamic="Required" />
  </Type>
  <Type Name="Decimal" Browse="Required Public" >
    <Method Name="ToString" Dynamic="Required" />
  </Type>
  <Type Name="Guid" Browse ="Required Public" >
    <Method Name="ToString" Dynamic="Required" />
  </Type>
  <Type Name="Int16" Browse="Required Public" >
    <Method Name="ToString" Dynamic="Required" />
  </Type>
  <Type Name="SByte" Browse="Required Public" >
    <Method Name="ToString" Dynamic="Required" />
  </Type>
  <Type Name="Single" Browse="Required Public" >
    <Method Name="ToString" Dynamic="Required" />
  </Type>
  <Type Name="TimeSpan" Browse="Required Public" >
    <Method Name="ToString" Dynamic="Required" />
  </Type>
  <Type Name="UInt16" Browse="Required Public" >
    <Method Name="ToString" Dynamic="Required" />
  </Type>
  <Type Name="UInt32" Browse="Required Public" >
    <Method Name="ToString" Dynamic="Required" />
  </Type>
  <Type Name="UInt64" Browse="Required Public" >
    <Method Name="ToString" Dynamic="Required" />
  </Type>
</Namespace>
</Application>
</Directives>

```

See also

- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Elements](#)

- [Runtime Directive Policy Settings](#)
 - [<MethodInstantiation> Element](#)
-

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

<MethodInstantiation> Element (.NET Native)

Article • 10/20/2022

Applies runtime reflection policy to a constructed generic method. See important info in **Remarks**.

Syntax

XML

```
<MethodInstantiation Name="method_name"
    Signature="method_signature"
    Arguments="method_arguments"
    Browse="policy_type"
    Dynamic="policy_type" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
<code>Name</code>	General	Required attribute. Specifies the method name.
<code>Signature</code>	General	Optional attribute. Specifies named parameters of the method. Multiple named parameters are separated by commas. The <code>Signature</code> attribute is used to differentiate overloaded methods.
<code>Arguments</code>	General	Required attribute. Specifies the generic type arguments. If multiple arguments are present, they are separated by commas.
<code>Browse</code>	Reflection	Optional attribute. Controls querying for information about or enumerating a method but does not enable any dynamic invocation at run time.
<code>Dynamic</code>	Reflection	Optional attribute. Controls runtime access to a constructor or method to enable dynamic programming. This policy ensures that a member can be invoked dynamically at run time.

Name attribute

Value	Description
<i>method_name</i>	The method name. The type of the method is defined by the parent <Type> or <TypeInstantiation> element.

Signature attribute

Value	Description
<i>method_signature</i>	Specifies the named parameters of the method. If multiple parameters are present, they are separated by commas.

Arguments attribute

Value	Description
<i>method_arguments</i>	Specifies the generic type arguments. If multiple arguments are present, they are separated by commas. Each argument must consist of the fully qualified type name.

All other attributes

Value	Description
<i>policy_setting</i>	The setting to apply to this policy type for the method. Possible values are <code>Auto</code> , <code>Excluded</code> , <code>Included</code> , and <code>Required</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

None.

Parent Elements

Element	Description
<Type>	Applies reflection policy to a type and all its members.
<TypeInstantiation>	Applies reflection policy to a constructed generic type and all its members.

Remarks

The `<MethodInstantiation>` element overrides the runtime reflection policy of its corresponding open generic method.

When you encounter a **System.Reflection.MissingRuntimeArtifactException** exception ('Cannot retrieve a MethodInfo for this delegate because the method it targeted [...] was not enabled for metadata using the Dynamic attribute.'), the message provides a url for you to visit, and that url takes you to the [MissingMetadataException troubleshooter](#) . Fill in the details in the **I reflect on** section, and then use the markup that's generated in the **Preview** section.

If there are no generic parameters, then leave **Comma-separated generic arguments to instantiate the method** blank. In that case you can disregard the **WARNING: The directive below is not complete yet** in the **Preview** section; and the **MethodInstantiation@Arguments** attribute will be empty in the **Preview**, by design. As documented above, the **Arguments** attribute specifies the generic type arguments, and not method parameters.

See also

- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Elements](#)
- [Runtime Directive Policy Settings](#)
- [<Method> Element](#)

<Namespace> Element (.NET Native)

Article • 10/20/2022

Applies runtime reflection policy to all the types in a specified namespace.

Syntax

XML

```
<Namespace Name="namespace_name"
  Activate="policy_type"
  Browse="policy_type"
  Dynamic="policy_setting"
  Serialize="policy_setting"
  DataContractSerializer="policy_setting"
  DataContractJsonSerializer="policy_setting"
  XmlSerializer="policy_setting"
  MarshalObject="policy_setting"
  MarshalDelegate="policy_setting"
  MarshalStructure="policy_setting" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
Name	General	Required attribute. Specifies the name of the namespace.
Activate	Reflection	Optional attribute. Controls runtime access to constructors to enable activation of instances.
Browse	Reflection	Optional attribute. Controls querying for information about program elements, but does not enable any runtime access.
Dynamic	Reflection	Optional attribute. Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming.

Attribute	Attribute type	Description
<code>Serialize</code>	Serialization	Optional attribute. Controls runtime access to constructors, fields, and properties, to enable type instances to be serialized and deserialized by libraries such as the Newtonsoft JSON serializer.
<code>DataContractSerializer</code>	Serialization	Optional attribute. Controls policy for serialization that uses the System.Runtime.Serialization.DataContractSerializer class.
<code>DataContractJsonSerializer</code>	Serialization	Optional attribute. Controls policy for JSON serialization that uses the System.Runtime.Serialization.Json.DataContractJsonSerializer class.
<code>XmlSerializer</code>	Serialization	Optional attribute. Controls policy for XML serialization that uses the System.Xml.Serialization.XmlSerializer class.
<code>MarshalObject</code>	Interop	Optional attribute. Controls policy for marshaling reference types to Windows Runtime and COM.
<code>MarshalDelegate</code>	Interop	Optional attribute. Controls policy for marshaling delegate types as function pointers to native code.
<code>MarshalStructure</code>	Interop	Optional attribute. Controls policy for marshaling structures to native code.

Name attribute

Value	Description
<i>namespace_name</i>	The namespace name. If the <Namespace> element is a child of an <Application> , <Library> , or <Assembly> element, <i>namespace_name</i> must be a fully qualified namespace name. If the <Namespace> element is a child of another <Namespace> element, <i>namespace_name</i> must be a relative namespace name.

All other attributes

Value	Description
<i>policy_setting</i>	The setting to apply to this policy type for all types in the namespace. Possible values are <code>All</code> , <code>Auto</code> , <code>Excluded</code> , <code>Public</code> , <code>PublicAndInternal</code> , <code>Required Public</code> , <code>Required PublicAndInternal</code> , and <code>Required All</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

Element	Description
<code><Namespace></code>	Applies runtime reflection policy to all types in a parent namespace.
<code><Type></code>	Applies reflection policy to a type.
<code><TypeInstantiation></code>	Applies reflection policy to a constructed generic type.

Parent Elements

Element	Description
<code><Application></code>	Serves as a container for application-wide types and type members whose metadata is available for reflection at run time. The <code><Application></code> element can have zero, one, or more <code><Assembly></code> elements.
<code><Assembly></code>	Applies runtime reflection policy to all the types in a specified assembly.
<code><Library></code>	Defines the assembly that contains types and type members whose metadata is available for reflection at run time. The <code><Library></code> element can have zero or one <code><Assembly></code> element.
<code><Namespace></code>	Applies reflection policy to all types in a parent namespace.

Remarks

The `Activate`, `Browse`, `Dynamic`, and `Serialize` attributes are all optional. If none are present, the `<Namespace>` element serves only as a container for child elements. If they are present, the `<Namespace>` element applies runtime reflection policy to all the types in the specified namespace.

When it is a child of the `<Assembly>` element, the `<Namespace>` element overrides the runtime reflection policy defined by the `<Assembly>` element.

See also

- [Runtime Directive Policy Settings](#)
- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Elements](#)

<Parameter> Element (.NET Native)

Article • 10/20/2022

Applies reflection policy to the type of the argument passed to a method.

Syntax

XML

```
<Parameter Name="parameter_name"
  Activate="policy_type"
  Browse="policy_type"
  Dynamic="policy_type"
  Serialize="policy_type"
  DataContractSerializer="policy_type"
  DataContractJsonSerializer="policy_type"
  XmlSerializer="policy_type"
  MarshalObject="policy_type"
  MarshalDelegate="policy_type"
  MarshalStructure="policy_type" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
Name	General	Required attribute. The parameter name. For example, for the method signature <code>String.CompareTo(Object value)</code> , the value of the <code>Name</code> attribute is "value".
Activate	Reflection	Optional attribute. Controls runtime access to constructors to enable activation of instances.
Browse	Reflection	Optional attribute. Controls querying for information about program elements, but does not enable any runtime access.

Attribute	Attribute type	Description
<code>Dynamic</code>	Reflection	Optional attribute. Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming.
<code>Serialize</code>	Serialization	Optional attribute. Controls runtime access to constructors, fields, and properties, to enable type instances to be serialized and deserialized by libraries such as the Newtonsoft JSON serializer.
<code>DataContractSerializer</code>	Serialization	Optional attribute. Controls policy for serialization that uses the System.Runtime.Serialization.DataContractSerializer class.
<code>DataContractJsonSerializer</code>	Serialization	Optional attribute. Controls policy for JSON serialization that uses the System.Runtime.Serialization.DataContractSerializer class.
<code>XmlSerializer</code>	Serialization	Optional attribute. Controls policy for XML serialization that uses the System.Xml.Serialization.XmlSerializer class.
<code>MarshalObject</code>	Interop	Optional attribute. Controls policy for marshaling reference types to WinRT and COM.
<code>MarshalDelegate</code>	Interop	Optional attribute. Controls policy for marshaling delegate types as function pointers to native code.
<code>MarshalStructure</code>	Interop	Optional attribute. Controls policy for marshaling value types to native code.

Name attribute

Value	Description
<i>parameter_name</i>	The name of the method parameter to which policy is applied. For example, for the method signature <code>String.CompareTo(Object value)</code> , the value of the <code>Name</code> attribute is "value".

All other attributes

Value	Description
<i>policy_setting</i>	The setting to apply to this policy type. Possible values are <code>All</code> , <code>Public</code> , <code>PublicAndInternal</code> , <code>Required Public</code> , <code>Required PublicAndInternal</code> , and <code>Required All</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

None.

Parent Elements

Element	Description
<Method>	Applies runtime reflection policy to a constructor or method.

Remarks

The `<Parameter>` element is a child of the [<Method>](#) element and is used to apply policy to a particular method parameter. The specific method parameter is specified by name rather than by type. At least one attribute that represents a policy type, such as `Activate` or `Dynamic`, must be present.

See also

- [<Method> Element](#)
- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Policy Settings](#)
- [Runtime Directive Elements](#)

<Property> Element (.NET Native)

Article • 10/20/2022

Applies runtime reflection policy to a property.

Syntax

XML

```
<Property Name="property_name"
  Browse="policy_type"
  Dynamic="policy_type"
  Serialize="policy_type" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
Name	General	Required attribute. Specifies the property name.
Browse	Reflection	Optional attribute. Controls querying for information about or enumerating the property but does not enable any dynamic access at run time.
Dynamic	Reflection	Optional attribute. Controls runtime access to the property to enable dynamic programming. This policy ensures that a property can be set or retrieved dynamically at run time.
Serialize	Serialization	Optional attribute. Controls runtime access to a property to enable type instances to be serialized by libraries such as the Newtonsoft JSON serializer or to be used for data binding.

Name attribute

Value	Description
-------	-------------

Value	Description
<i>method_name</i>	The property name. The type of the property is defined by the parent <Type> or <TypeInstantiation> element.

All other attributes

Value	Description
<i>policy_setting</i>	The setting to apply to this policy type for the property. Possible values are <code>Auto</code> , <code>Excluded</code> , <code>Included</code> , and <code>Required</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

None.

Parent Elements

Element	Description
<Type>	Applies reflection policy to a type and all its members.
<TypeInstantiation>	Applies reflection policy to a constructed generic type and all its members.

Remarks

If a property's policy is not explicitly defined, it inherits the runtime policy of its parent element.

Example

The following example uses reflection to instantiate a `Book` object and display its property values. The original default.rd.xml file for the project appears as follows:

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
    <Namespace Name="LibraryApplications" Browse="Required Public" >
      <Type Name="Book" Activate="All" />
    </Namespace>
  </Application>
</Directives>
```

```
</Application>
</Directives>
```

The file applies the `All` value to the `Activate` policy for the `Book` class, which allows access to class constructors through reflection. The `Browse` policy for the `Book` class is inherited from its parent namespace. This is set to `Required Public`, which makes metadata available at runtime.

The following is the source code for the example. The `outputBlock` variable represents a `TextBlock` control.

C#

```
namespace LibraryApplications
{
    public sealed class Example
    {
        public void Execute()
        {
            TextBlock outputBlock = MainPage.outputBlock;
            Type t = typeof(Book);
            Object obj = Activator.CreateInstance(t, new Object[] { "A Tale of
2 Cities", "Charles Dickens", "" });
            outputBlock.Text += "\n\n\n";
            foreach (var p in t.GetRuntimeProperties())
            {
                outputBlock.Text += String.Format("{0}: {1}\n", p.Name,
p.GetValue(obj));
            }
        }
    }

    public class Book
    {
        private string bookTitle = "";
        private string bookAuthor = "";
        private string bookISBN = "";

        public Book(string title, string author, string isbn)
        {
            bookTitle = title;
            bookAuthor = author;
            bookISBN = isbn;
        }

        public string Title
        {
            get { return bookTitle; }
            set { bookTitle = value; }
        }
    }
}
```

```

    public string Author
    {
        get { return bookAuthor; }
        set { bookTitle = value; }
    }

    public string ISBN
    {
        get { return bookISBN; }
    }

    public override string ToString()
    {
        return String.Format("{0}, {1}", Author, Title);
    }
}

public class TextUtilities
{
    public static string ConvertNumberToWord(int value)
    {
        switch (value)
        {
            case 1:
                return "One";
            case 2:
                return "Two";
            case 3:
                return "Three";
            case 4:
                return "Four";
            case 5:
                return "Five";
            case 6:
                return "Six";
            case 7:
                return "Seven";
            case 8:
                return "Eight";
            case 9:
                return "Nine";
            default:
                return value.ToString();
        }
    }
}

```

However, compiling and executing this example throws a [MissingRuntimeArtifactException](#) exception. Although we've made metadata for the `Book` type available, we've failed to make the implementations of the property getters available dynamically. We can correct this error by either in one of two ways:

- by defining the `Dynamic` policy for the `Book` type in its `<Type>` element.
- By adding a nested `<Property>` element for each property whose getter we'd like to invoke, as the following `default.rd.xml` file does.

XML

```
<Directives
  xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
    <Namespace Name="LibraryApplications" Browse="Required Public" >
      <Type Name="Book" Activate="All" >
        <Property Name="Title" Dynamic="Required" />
        <Property Name="Author" Dynamic="Required" />
        <Property Name="ISBN" Dynamic="Required" />
      </Type>
    </Namespace>
  </Application>
</Directives>
```

See also

- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Elements](#)
- [Runtime Directive Policy Settings](#)

<Subtypes> Element (.NET Native)

Article • 10/20/2022

Applies runtime policy to all classes inherited from the containing type.

Syntax

XML

```
<Subtypes Activate="policy_type"
           Browse="policy_type"
           Dynamic="policy_type"
           Serialize="policy_type"
          DataContractSerializer="policy_setting"
           DataContractJsonSerializer="policy_setting"
           XmlSerializer="policy_setting"
           MarshalObject="policy_setting"
           MarshalDelegate="policy_setting"
           MarshalStructure="policy_setting" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
Activate	Reflection	Optional attribute. Controls runtime access to constructors to enable activation of instances.
Browse	Reflection	Optional attribute. Controls querying for information about program elements, but does not enable any runtime access.
Dynamic	Reflection	Optional attribute. Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming.
Serialize	Serialization	Optional attribute. Controls runtime access to constructors, fields, and properties, to enable type instances to be serialized and deserialized by libraries such as the Newtonsoft JSON serializer.

Attribute	Attribute type	Description
<code>DataContractSerializer</code>	Serialization	Optional attribute. Controls policy for serialization that uses the System.Runtime.Serialization.DataContractSerializer class.
<code>DataContractJsonSerializer</code>	Serialization	Optional attribute. Controls policy for JSON serialization that uses the System.Runtime.Serialization.Json.DataContractJsonSerializer class.
<code>XmlSerializer</code>	Serialization	Optional attribute. Controls policy for XML serialization that uses the System.Xml.Serialization.XmlSerializer class.
<code>MarshalObject</code>	Interop	Optional attribute. Controls policy for marshaling reference types to Windows Runtime and COM.
<code>MarshalDelegate</code>	Interop	Optional attribute. Controls policy for marshaling delegate types as function pointers to native code.
<code>MarshalStructure</code>	Interop	Optional attribute. Controls policy for marshaling value types to native code.

All attributes

Value	Description
<code>policy_setting</code>	The setting to apply to this policy type. Possible values are <code>All</code> , <code>Auto</code> , <code>Excluded</code> , <code>Public</code> , <code>PublicAndInternal</code> , <code>Required Public</code> , <code>Required PublicAndInternal</code> , and <code>Required All</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

None.

Parent Elements

Element	Description
<Type>	Applies reflection policy to a type and all its members.

Remarks

The `<Subtypes>` element applies policy to all the subtypes of its containing type. You use it when you want to apply different policies to derived types and their base classes.

The reflection, serialization, and interop attributes are all optional, although at least one should be present.

Example

The following example defines a class named `BaseClass` and a subclass named `Derived1`.

C#

```
namespace Examples.Libraries
{
    public class BaseClass
    {
        public BaseClass()
        { }

        public override string ToString()
        {
            return String.Format("{0} Version {1}", this.GetType().Name,
Version);
        }
        public virtual double Version
        { get { return 1.0; }}
    }

    public class Derived1 : BaseClass
    {
        public Derived1() : base()
        {}

        public override double Version
        { get { return 1.1; }}

        public override string ToString()
        {
            return String.Format("{0} Version {1}", this.GetType().Name,
Version);
        }
    }
}
```

As shown in the following code, the runtime directives file explicitly sets the `Dynamic` and `Activate` policies for `BaseClass` to `Excluded`. Because of this, objects of type `BaseClass` cannot be instantiated dynamically or by calls to the `BaseClass` class constructor.

However, the `<Subtypes>` element allows classes derived from `BaseClass` to be instantiated dynamically and through calls to their class constructors.

XML

```
<Directives xmlns="http://schemas.microsoft.com/netfx/2013/01/metadata">
  <Application>
    <Assembly Name="*Application*" Dynamic="Required All" />
    <Type Name="Examples.Libraries.BaseClass" Activate="Excluded"
Dynamic="Excluded" >
      <Subtypes Activate="Public" Dynamic="Public"/>
    </Type>
  </Application>
</Directives>
```

Because of the `<Subtypes>` directive, the following code that instantiates a `Derived1` instance dynamically by calling the `Activator.CreateInstance(Type)` method executes successfully. The block variable here is a `TextBlock` object in a blank UWP app.

C#

```
namespace Examples
{
    using System.Reflection;
    using Examples.Libraries;

    public sealed partial class MainPage : Page
    {
        public MainPage()
        {
            this.InitializeComponent();

            Derived1 d1 = new Derived1();
            block.Text += d1.ToString() + Environment.NewLine;

            Type derivedType = typeof(Derived1);
            Object obj = Activator.CreateInstance(derivedType);
            block.Text += obj.GetType().FullName + Environment.NewLine;
        }
    }
}
```

See also

- [<Type> Element](#)
- [Runtime Directives \(rd.xml\) Configuration File Reference](#)
- [Runtime Directive Elements](#)

- [Runtime Directive Policy Settings](#)

<Type> Element (.NET Native)

Article • 10/20/2022

Applies runtime policy to a particular type, such as a class or structure.

Syntax

XML

```
<Type Name="type_name"
      Activate="policy_type"
      Browse="policy_type"
      Dynamic="policy_type"
      Serialize="policy_type"
      DataContractSerializer="policy_setting"
      DataContractJsonSerializer="policy_setting"
      XmlSerializer="policy_setting"
      MarshalObject="policy_setting"
      MarshalDelegate="policy_setting"
      MarshalStructure="policy_setting" />
```

Attributes and Elements

The following sections describe attributes, child elements, and parent elements.

Attributes

Attribute	Attribute type	Description
Name	General	Required attribute. Specifies the type name.
Activate	Reflection	Optional attribute. Controls runtime access to constructors to enable activation of instances.
Browse	Reflection	Optional attribute. Controls querying for information about program elements, but does not enable any runtime access.
Dynamic	Reflection	Optional attribute. Controls runtime access to all type members, including constructors, methods, fields, properties, and events, to enable dynamic programming.

Attribute	Attribute type	Description
<code>Serialize</code>	Serialization	Optional attribute. Controls runtime access to constructors, fields, and properties, to enable type instances to be serialized and deserialized by libraries such as the Newtonsoft JSON serializer.
<code>DataContractSerializer</code>	Serialization	Optional attribute. Controls policy for serialization that uses the System.Runtime.Serialization.DataContractSerializer class.
<code>DataContractJsonSerializer</code>	Serialization	Optional attribute. Controls policy for JSON serialization that uses the System.Runtime.Serialization.Json.DataContractJsonSerializer class.
<code>XmlSerializer</code>	Serialization	Optional attribute. Controls policy for XML serialization that uses the System.Xml.Serialization.XmlSerializer class.
<code>MarshalObject</code>	Interop	Optional attribute. Controls policy for marshaling reference types to Windows Runtime and COM.
<code>MarshalDelegate</code>	Interop	Optional attribute. Controls policy for marshaling delegate types as function pointers to native code.
<code>MarshalStructure</code>	Interop	Optional attribute. Controls policy for marshaling value types to native code.

Name attribute

Value	Description
<i>type_name</i>	The type name. If this <code><Type></code> element is the child of either a <code><Namespace></code> element or another <code><Type></code> element, <i>type_name</i> can include the name of the type without its namespace. Otherwise, <i>type_name</i> must include the fully qualified type name.

All other attributes

Value	Description
<i>policy_setting</i>	The setting to apply to this policy type. Possible values are <code>All</code> , <code>Auto</code> , <code>Excluded</code> , <code>Public</code> , <code>PublicAndInternal</code> , <code>Required Public</code> , <code>Required PublicAndInternal</code> , and <code>Required All</code> . For more information, see Runtime Directive Policy Settings .

Child Elements

Element	Description
<AttributeImplies>	If the containing type is an attribute, defines runtime policy for code elements to which the attribute is applied.
<Event>	Applies reflection policy to an event belonging to this type.
<Field>	Applies reflection policy to a field belonging to this type.
<GenericParameter>	Applies policy to the parameter type of a generic type.
<ImpliesType>	Applies policy to a type, if that policy has been applied to the type represented by the containing <Type> element.
<Method>	Applies reflection policy to a method belonging to this type.
<MethodInstantiation>	Applies reflection policy to a constructed generic method belonging to this type.
<Property>	Applies reflection policy to a property belonging to this type.
<Subtypes>	Applies runtime policy to all classes inherited from the containing type.
<Type>	Applies reflection policy to a nested type.
<TypeInstantiation>	Applies reflection policy to a constructed generic type.

Parent Elements

Element	Description
<Application>	Serves as a container for application-wide types and type members whose metadata is available for reflection at run time.
<Assembly>	Applies reflection policy to all the types in a specified assembly.
<Library>	Defines the assembly that contains types and type members whose metadata is available for reflection at run time.
<Namespace>	Applies reflection policy to all the types in a namespace.
<Type>	Applies reflection policy to a type and all its members.
<TypeInstantiation>	Applies reflection policy to a constructed generic type and all its members.

Remarks

The reflection, serialization, and interop attributes are all optional. If none are present, the `<Type>` element serves as a container whose child types define policy for individual members.

If a `<Type>` element is the child of an `<Assembly>`, `<Namespace>`, `<Type>`, or `<TypeInstantiation>` element, it overrides the policy settings defined by the parent element.

A `<Type>` element of a generic type applies its policy to all instantiations that do not have their own policy. The policy of constructed generic types is defined by the `<TypeInstantiation>` element.

If the type is a generic type, its name is decorated by a grave accent symbol (```) followed by its number of generic parameters. For example, the `Name` attribute of a `<Type>` element for the `System.Collections.Generic.List<T>` class appears as

```
Name="System.Collections.Generic.List`1".
```

Example 1

The following example uses reflection to display information about the fields, properties, and methods of the `System.Collections.Generic.List<T>` class. The variable `b` in the example is a `TextBlock` control. Because the example simply retrieves type information, the availability of metadata is controlled by the `Browse` policy setting.

C#

```
public static void GetReflectionInfo()
{
    Type t = typeof(List<>);
    b.Text += String.Format("Type information for {0}\n", t);

    // Get fields.
    b.Text += "\nFields:\n";

    var fields = t.GetTypeInfo().DeclaredFields;
    int nFields = 0;
    foreach (var field in fields)
    {
        b.Text += String.Format("    {0} ({1})", field.Name,
field.FieldType.Name);
        nFields++;
    }
    if (nFields == 0) b.Text += "    None\n";

    // Get properties.
    b.Text += "\nProperties:\n";
    var props = t.GetTypeInfo().DeclaredProperties;
```